perplexity

Напиши код на языке пайтон для моей нейросети

Вот готовый код нашей модели Инь–Ян на Python: он строит фазовый портрет, рассчитывает движение системы и сохраняет результаты в CSV. Его можно передать твоей нейросети для запуска или дальнейшей доработки.

Это **симулятор** математической модели, а не архитектура самой нейросети.

Код

Нужны библиотеки `numpy` и `matplotlib`:

```
pip install numpy matplotlib
```

```
import numpy as np
import matplotlib.pyplot as plt


def dynamics(state):
    """Правая часть системы: dx/dt, dv/dt."""
    x, v = state
    return np.array([
        (1.0 - x**2) * v,
        -x
    ], dtype=float)


def invariant(x, v):
    """Сохраняющаяся величина H. Определена при |x| < 1."""
    return 0.5 * v**2 - 0.5 * np.log1p(-x**2)


def rk4_step(state, dt):
    """Один шаг метода Рунге — Кутты четвёртого порядка."""
    k1 = dynamics(state)
    k2 = dynamics(state + 0.5 * dt * k1)
    k3 = dynamics(state + 0.5 * dt * k2)
    k4 = dynamics(state + dt * k3)

    return state + dt * (k1 + 2*k2 + 2*k3 + k4) / 6.0


def simulate(x0=0.0, v0=0.8, duration=40.0, dt=0.01):
    """Рассчитать траекторию и вернуть массивы времени и состояний."""
    if not (-1.0 < x0 < 1.0):
        raise ValueError("Начальное x0 должно находиться между -1 и 1.")
```

```
    if duration <= 0 or dt <= 0:
        raise ValueError("duration и dt должны быть положительными.")

    steps = int(np.ceil(duration / dt))
    t = np.linspace(0.0, duration, steps + 1)
    actual_dt = t[^1] - t[^0]

    states = np.empty((steps + 1, 2), dtype=float)
    states[^0] = [x0, v0]

    for i in range(steps):
        states[i + 1] = rk4_step(states[i], actual_dt)

        if not np.all(np.isfinite(states[i + 1])):
            raise RuntimeError("Численное решение неустойчиво: уменьшите dt.")

        if abs(states[i + 1, 0]) >= 1.0:
            raise RuntimeError(
                "Решение вышло за область |x| < 1. Уменьшите dt."
            )

    return t, states


def draw_phase_portrait(ax, states):
    """Поле направлений, точные линии уровня H и одна траектория."""
    grid_x = np.linspace(-0.98, 0.98, 45)
    grid_v = np.linspace(-2.2, 2.2, 55)
    X, V = np.meshgrid(grid_x, grid_v)

    U = (1.0 - X**2) * V
    W = -X

    ax.streamplot(
        grid_x, grid_v, U, W,
        color="#b8bec6",
        density=0.8,
        linewidth=0.6,
        arrowsize=1.0
    )

    levels = [0.03, 0.12, 0.30, 0.65, 1.20, 2.0]
    colors = plt.cm.viridis(np.linspace(0.1, 0.9, len(levels)))

    # Параметризация точных замкнутых линий H = const.
    theta = np.linspace(0.0, 2.0 * np.pi, 1200)

    for h, color in zip(levels, colors):
        v_curve = np.sqrt(2.0 * h) * np.sin(theta)
        q = np.maximum(2.0 * h - v_curve**2, 0.0)
        x_curve = np.sign(np.cos(theta)) * np.sqrt(-np.expm1(-q))

        ax.plot(
            x_curve, v_curve,
            color=color,
```

```
            linewidth=1.4,
            label=f"H = {h:g}"
        )

    ax.plot(
        states[:, 0], states[:, 1],
        color="crimson",
        linewidth=2.0,
        label="Selected trajectory"
    )

    ax.scatter(
        states[0, 0], states[0, 1],
        color="crimson", marker="o", s=45, zorder=5
    )
    ax.scatter(0, 0, color="black", s=30, zorder=5)

    ax.axvline(-1, color="gray", linestyle="--")
    ax.axvline(1, color="gray", linestyle="--")
    ax.axhline(0, color="gray", linewidth=0.5)
    ax.axvline(0, color="gray", linewidth=0.5)

    ax.set(
        xlim=(-1.05, 1.05),
        ylim=(-2.25, 2.25),
        xlabel="x = Yang - Yin",
        ylabel="v",
        title="Phase portrait: clockwise motion"
    )
    ax.legend(fontsize=8, loc="upper right")
    ax.grid(alpha=0.15)


def main():
    # Измени эти параметры для своего эксперимента.
    x0 = 0.0
    v0 = 0.8
    duration = 40.0
    dt = 0.01

    t, states = simulate(x0, v0, duration, dt)
    x, v = states.T

    yang = (1.0 + x) / 2.0
    yin = (1.0 - x) / 2.0
    h = invariant(x, v)

    data = np.column_stack([t, x, v, yin, yang, h])

    np.savetxt(
        "yin_yang_data.csv",
        data,
        delimiter=",",
        header="time,x,v,yin,yang,H",
        comments=""
    )
```

```
    fig, axes = plt.subplots(
        1, 2,
        figsize=(14, 6),
        constrained_layout=True
    )

    draw_phase_portrait(axes[^0], states)

    axes[^1].plot(t, yin, color="#384454", label="Yin")
    axes[^1].plot(t, yang, color="#e5a323", label="Yang")
    axes[^1].axhline(0.5, color="gray", linestyle="--", linewidth=0.8)
    axes[^1].set(
        xlabel="Time",
        ylabel="Share",
        ylim=(0, 1),
        title="Alternation of Yin and Yang"
    )
    axes[^1].legend()
    axes[^1].grid(alpha=0.2)

    fig.savefig("yin_yang_model.png", dpi=200)
    fig.savefig("yin_yang_model.pdf")

    print(f"Initial H: {h[^0]:.8f}")
    print(f"Maximum numerical drift in H: {np.max(np.abs(h - h[^0])):.3e}")

    plt.show()


if __name__ == "__main__":
    main()
```

Что получишь

- Слева — фазовый портрет и выбранную траекторию.
- Справа — чередование долей Инь и Ян во времени.
- PNG и PDF с графиками.
- CSV с колонками `time, x, v, yin, yang, H`, который можно использовать как исходные данные для обучения.

Для задачи предсказания состояния пары удобно сформировать так:

```
inputs = states[:-1]    # Состояние [x, v] в текущий момент.
targets = states[1:]    # Состояние [x, v] на следующем шаге.
```

Здесь нейросеть будет учиться отображению

$$(x_t, v_t) \longmapsto (x_{t+\Delta t}, v_{t+\Delta t}).$$

Для обучения на разных режимах следует сгенерировать несколько траекторий с разными начальными условиями. В обучающую и тестовую выборки лучше разносить

целые траектории, а не соседние точки одной и той же кривой.

⁂